

Variational Quantum EigenSolver

A beginner-friendly guide

1 Beginning

This guide introduces you to one of the most exciting and practical algorithms in the realm of quantum computing, Variational Quantum Eigensolver or VQE. VQE is special because it is designed to run on the noisy, small-scale quantum hardware that we have today. This makes it one of the most actively researched algorithms in the field right now.

The only prerequisite is a basic understanding of qubits and quantum gates. Everything else will be introduced as we need it.

2 Background

i) What is an Eigenvalue?

Before we can understand VQE, we need to understand what problem it is solving. That problem involves something called an **eigenvalue**.

Do not let the word intimidate you. The idea is quite natural. Imagine you have a mathematical transformation — something that takes a vector and produces another vector. Most of the time, the output vector points in a completely different direction to the input.

However, there are special vectors called **eigenvectors** for which the transformation only stretches or shrinks the vector, never rotating it. The amount by which it is stretched or shrunk is called the **eigenvalue**. In quantum mechanics, the transformation we care about is the **Hamiltonian** — a mathematical operator that describes the total energy of a quantum system. The eigenvalues of the Hamiltonian are the allowed energy levels of that system.

ii) Why Do Eigenvalues Matter in Quantum Chemistry?

Here is where things get exciting. In quantum chemistry, we are often interested in molecules and their properties — things like bond lengths, reaction energies, and whether a drug molecule will bind to a protein.

All of these properties are ultimately determined by the **ground state energy** of the molecule: the lowest possible energy that the molecule can have. Chemists and

materials scientists desperately need accurate ground state energies to design new drugs, batteries, fertilisers, and materials.

The ground state energy is simply the smallest eigenvalue of the molecular Hamiltonian. So the question becomes:

The Central Problem

Given a molecule described by a Hamiltonian H , find its lowest energy eigenvalue (the ground state energy) as accurately and efficiently as possible. This is the problem that VQE is designed to solve.

iii) The Challenge of Classical Computation

You might wonder: why not just solve this on a regular computer? The answer is that classical systems grow exponentially complex. To describe a system of n electrons exactly on a classical computer, the amount of memory required grows as 2^n . For just 50 electrons, that is more memory than all the classical computers in the world combined. This is known as the **exponential wall**.

Approximate classical methods exist (such as density functional theory), but they can fail badly for strongly correlated systems — exactly the molecules that are most interesting and hardest to understand.

This is where quantum computers offer a natural advantage. A quantum computer with n qubits can naturally represent and manipulate 2^n quantum states simultaneously, suggesting it might be able to handle this exponential complexity far more efficiently.

3 The Variational Quantum Eigensolver

i) The Problem: Finding Ground State Energy

Consider a simple molecule: molecular hydrogen, H_2 . This is the simplest molecule imaginable — just two protons and two electrons. Yet even for this tiny system, computing the exact ground state energy classically becomes challenging as we demand greater precision.

For larger molecules relevant to drug discovery or materials science, exact classical methods become completely infeasible. A quantum computer, however, can represent the quantum state of such a system much more compactly.

The challenge then is: how do we get the quantum computer to find the lowest energy state? This is the job of VQE.

ii) Importance

VQE was first proposed in 2014 by Peruzzo et al., and it quickly became one of the most important algorithms in quantum chemistry and materials science. Here is why it matters so much:

- **Drug discovery:** Accurately computing molecular energies could accelerate the design of new medicines by predicting how molecules interact.
- **Battery technology:** Better simulation of chemical reactions could lead to dramatically improved lithium-ion or solid-state batteries.
- **Fertiliser production:** The Haber-Bosch process for producing nitrogen fertiliser consumes about 1-2% of the world's energy. A quantum simulation of the nitrogenase enzyme — which does the same job at room temperature — could revolutionise agriculture.
- **Materials science:** Designing new superconductors, catalysts, and semiconductors all require accurate quantum simulations that classical computers struggle with.

The reason VQE receives so much attention is that, unlike Shor's Algorithm, it can run on today's imperfect quantum hardware. It is designed specifically to work with noise and limited qubit counts.

iii) VQE

The key idea behind VQE is elegantly simple:

The Core Idea

Instead of finding the ground state directly — which would require a perfect quantum computer — VQE uses an educated guess for the quantum state and then repeatedly improves that guess using classical optimization.

The quantum computer measures the energy. The classical computer updates the guess. Together, they converge on the ground state energy.

This is what the name means. The algorithm is:

- **Variational** — it uses the variational principle (explained next).
- **Quantum** — a quantum computer evaluates the energy.
- **Eigensolver** — it finds an eigenvalue (the ground state energy) of the Hamiltonian.

The hybrid nature — quantum for measurement, classical for optimization — is what makes VQE practical on today's hardware, which is too noisy for fully quantum algorithms.

iv) The Variational Principle

The foundation of VQE is a mathematical fact from quantum mechanics called the **variational principle**. It states:

The Variational Principle

For any quantum state $|\psi\rangle$, the expectation value of the Hamiltonian H is always greater than or equal to the true ground state energy, E_0

In other words, any state we guess will give an energy that is at least as high as the true ground state energy. It can never go below it. This is a remarkably powerful statement. It tells us something crucial about our strategy:

- We can try any quantum state we like as a guess.
- We measure its energy using the Hamiltonian.
- We then try to find a state that gives a *lower* energy.
- The lowest energy we find is our best approximation to the true ground state.

The better our guess, the closer we get to E_0 . The variational principle guarantees we can never accidentally cheat our way to an energy that is too low.

Think of it like trying to find the bottom of a valley while blindfolded. You cannot step below the valley floor — that is guaranteed by physics. You can only move to lower and lower ground. Eventually, if you explore carefully enough, you reach the bottom.

$$E(\theta) \geq E_0$$

The energy E depends on the parameter, θ . VQE minimizes $E(\theta)$ over all possible parameter values.

v) The Ansatz

Now comes an important question: what quantum states should we try? We cannot try every possible state — there are infinitely many of them. We need a smarter strategy.

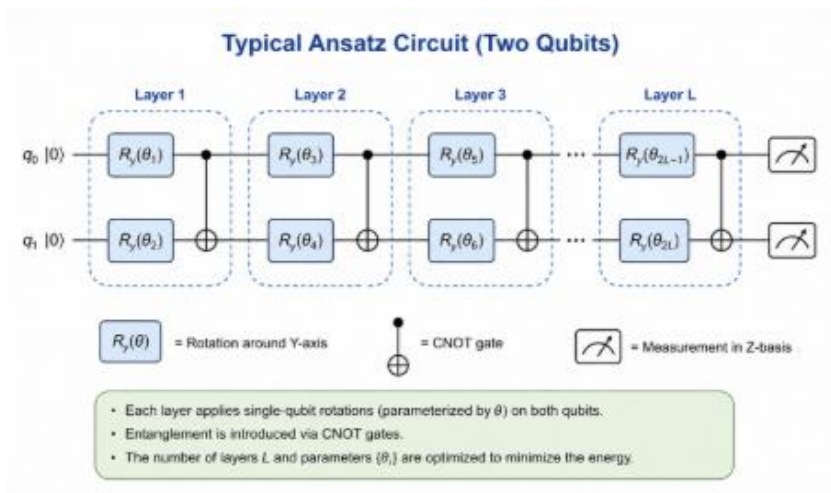
The answer is the **ansatz** (from German, meaning 'approach' or 'starting point'). An ansatz is a parameterized family of quantum states — a quantum circuit whose gates have adjustable angles, called parameters.

Here is a simple example. Suppose we have a single qubit. A parameterized state might look like:

$$|\psi(\theta)\rangle = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle$$

By adjusting the angle theta, we can sweep through a range of quantum states. For a real molecule, we use many qubits and many parameters, but the principle is the same.

A typical ansatz circuit for VQE looks like this:



Choosing a good ansatz is both an art and a science. The ansatz must be:

- **Expressive enough** to contain (or closely approximate) the true ground state.
- **Simple enough** to run on a real quantum computer without accumulating too much noise.

A popular choice is the **Hardware-Efficient Ansatz (HEA)**, which is designed to match the native gate set of a specific quantum processor, minimising noise. Another common choice is the **Unitary Coupled Cluster (UCC)** ansatz, which is motivated by chemistry and tends to be more accurate.

4 The Classical Optimizer

i) How Optimization Works

Once we have an ansatz with adjustable parameters, we need a strategy for choosing the best parameter values. This is the job of the **classical optimizer**.

The optimizer works like this. We start with some initial guess for the parameters — perhaps just random angles. We run the quantum circuit, measure the energy, and then ask: which direction should we change the parameters to make the energy lower?

After each energy measurement, the classical optimizer updates the parameters and sends the new values back to the quantum computer. This loop continues until the energy stops decreasing — meaning we have found the minimum.

ii) The Energy Landscape

To understand what the optimizer is doing, it helps to imagine an **energy landscape**. Think of a hilly terrain where the height at any point represents the energy for that combination of parameter values.

Our goal is to find the lowest valley in this landscape — the global minimum. The optimizer explores this landscape step by step, always trying to move downhill.

Here is a simplified view of the energy as a function of a single parameter theta:

Parameter (theta)	Energy E(theta)	Notes
0.0	0.50	Starting point (local region)
0.5	0.35	Energy decreasing
1.0	0.18	Approaching minimum
1.57	-0.48	Near minimum
2.0	-0.42	Slightly past minimum
pi	-0.50	Global minimum found

iii) Common Optimizers

Several optimization algorithms are used in VQE. Each has trade-offs:

- **COBYLA** (Constrained Optimization BY Linear Approximation) — does not require gradients. Very practical for noisy hardware because small measurement errors do not derail the optimization. Widely used for VQE experiments.
- **SPSA** (Simultaneous Perturbation Stochastic Approximation) — estimates the gradient using only two circuit evaluations regardless of the number of parameters. Highly efficient and robust to noise.

- **Parameter-shift rule** — computes exact gradients of a quantum circuit by evaluating it at shifted parameter values. More precise than SPSA but requires more circuit evaluations.

For beginners, COBYLA is usually the recommended starting point because it is simple, requires no gradient computation, and works reasonably well on noisy devices.

5 VQE Step by Step

Now that we have all the ingredients, let us walk through VQE from start to finish using molecular hydrogen H_2 as our example.

Step 1: Define the molecule. We choose H_2 at a bond length of 0.735 Angstroms (its equilibrium geometry). This molecule has 2 electrons in 4 spin-orbitals, which maps to a 2-qubit problem after applying symmetry reductions.

Step 2: Build the Hamiltonian. Using quantum chemistry methods (handled by classical software), we construct the qubit Hamiltonian for H_2 . It is expressed as a sum of Pauli operators — things like Z , X , Y , ZZ , ZI — each with a coefficient:

$$H = g_0 I + g_1 Z_0 + g_2 Z_1 + g_3 Z_0 Z_1 + g_4 X_0 X_1 + g_5 Y_0 Y_1.$$

Each coefficient g_i is calculated using classical quantum chemistry software, while the Pauli operators (X , Y , and Z) describe how the qubits interact.

Step 3: Choose an ansatz. For H_2 , a simple 2-qubit ansatz suffices. We apply a rotation gate $R_y(\theta)$ on each qubit followed by an entangling CNOT gate. This gives us one or two free parameters to optimize.

Step 4: Measure the energy. We prepare the state $|\psi(\theta)\rangle$ on the quantum computer, then measure the expectation value of each Pauli term in the Hamiltonian. The total energy is the weighted sum of all these measurements.

Step 5: Optimize. We pass the measured energy to the classical optimizer, which proposes new parameter values. We repeat from Step 4.

Step 6: Extract the result. When the energy converges (stops decreasing), we report the minimum energy found as our approximation to the ground state energy of H₂. For H₂, the exact ground state energy is approximately -1.137 Hartree. A well-tuned VQE should get very close to this value:

Method	Ground State Energy (Hartree)	Notes
Exact (Full CI)	-1.1373	Reference value
VQE (ideal simulator)	-1.1372	< 1 mHartree error
VQE (noisy hardware)	-1.124	Slight noise penalty
Classical HF theory	-1.1175	Less accurate

6 Implementing VQE

Now it is time to see VQE in code. We will use **Qiskit** and the **qiskit-nature** package, which provides tools for quantum chemistry. Modern frameworks handle much of the heavy lifting, letting us focus on understanding the algorithm rather than reimplementing quantum chemistry from scratch.

Trying finding answers to these questions during your implementation as this would help connect your intuition to the implementation.

- Where is the Hamiltonian being built?
- Where is the ansatz circuit being constructed?
- Where is the energy being measured?
- Where is the classical optimizer running?
- Where is the final result extracted?

i) Choosing the Molecule

The first step is to define the molecule we want to simulate. We specify its atoms, their positions in space, and the charge and spin multiplicity.

```

from qiskit_nature.second_q.drivers import PySCFDriver
from qiskit_nature.second_q.mappers import JordanWignerMapper

# Define H2 at equilibrium bond length
driver = PySCFDriver(
    atom='H 0 0 0; H 0 0 0.735', # Two H atoms, 0.735 Angstrom apart
    basis='sto3g',                # Minimal basis set for orbital calculation
    charge=0,                     # Neutral molecule
    spin=0,                       # Singlet state (no unpaired electrons)
)

problem = driver.run()            # Run the classical quantum chemistry step
print(f'Number of particles: {problem.num_particles}') # Expected: (1, 1)
print(f'Number of spatial orbitals: {problem.num_spatial_orbitals}')


```

```

Number of particles: (1, 1)
Number of spatial orbitals: 2

```

ii) Building the Hamiltonian

Next, we convert the molecular Hamiltonian into a form that a quantum computer can process. This involves mapping fermionic operators (which describe electrons) to qubit operators (Pauli matrices). The **Jordan-Wigner mapping** is the most common choice.

```

mapper = JordanWignerMapper()

# Convert the fermionic Hamiltonian to a qubit Hamiltonian
qubit_op = mapper.map(problem.second_q_ops()[0])

print(f'\nNumber of qubits needed: {qubit_op.num_qubits}')
print(f'Number of Pauli terms: {len(qubit_op)}')

```

```

Number of qubits needed: 4
Number of Pauli terms: 15

```

iii) Constructing the Ansatz Circuit

Now we build the parameterized quantum circuit — the ansatz. Qiskit provides several ready-made ansatzes. We use **UCCSD** (Unitary Coupled Cluster Singles and Doubles), which is chemistry-inspired and typically very accurate.

```

from qiskit_nature.second_q.circuit.library import UCCSD, HartreeFock

# Reference state: the Hartree-Fock (classical best-guess) state
hf_state = HartreeFock(
    num_spatial_orbitals=problem.num_spatial_orbitals,
    num_particles=problem.num_particles,
    qubit_mapper=mapper,
)

# Build the UCCSD ansatz on top of the Hartree-Fock reference
ansatz = UCCSD(
    num_spatial_orbitals=problem.num_spatial_orbitals,
    num_particles=problem.num_particles,
    qubit_mapper=mapper,
    initial_state=hf_state,
)

print(f'\nNumber of variational parameters: {ansatz.num_parameters}')
# Expected: 3 (3 free angles for the optimizer to tune)

```

```

Number of variational parameters: 3

```

iv) Measuring the Energy

This is the heart of VQE. For a given set of parameter values, we prepare the state $|\psi(\theta)\rangle$ and measure the expectation value of the Hamiltonian. This is done by measuring each Pauli term separately and summing the results:

```

from qiskit.primitives import StatevectorEstimator

estimator = StatevectorEstimator()

# Evaluate energy for a specific set of initial parameters (as a sanity check)
theta_init = [0.1, -0.2, 0.05] # Example starting parameters
job = estimator.run([(ansatz, qubit_op, theta_init)])
energy_init = float(job.result()[0].data.evs)

print(f"\nEnergy at initial parameters (all zeros) : {energy_init:.6f} Hartree")
print(" (This is the Hartree-Fock energy – not yet optimized)")

```

```

Energy at initial parameters (all zeros) : -1.794807 Hartree
(This is the Hartree-Fock energy – not yet optimized)

```

In Qiskit, the `StatevectorEstimator` primitive handles this automatically. You simply pass it the circuit and the Hamiltonian, and it returns the energy:

```

from qiskit_algorithms import VQE
from qiskit_algorithms.optimizers import COBYLA

# Set up the optimizer
optimizer = COBYLA(maxiter=500) # Maximum 500 function evaluations

# Create the VQE algorithm object
vqe = VQE(estimator, ansatz, optimizer)

# Run VQE – this executes the full optimization loop
result = vqe.compute_minimum_eigenvalue(qubit_op)

```

v) Running the Classical Optimizer

Now we put the whole loop together. The VQE Result object stores the optimal parameters and the corresponding ground state energy:

```

EXACT_ENERGY = -1.137276 # Full Configuration Interaction reference value

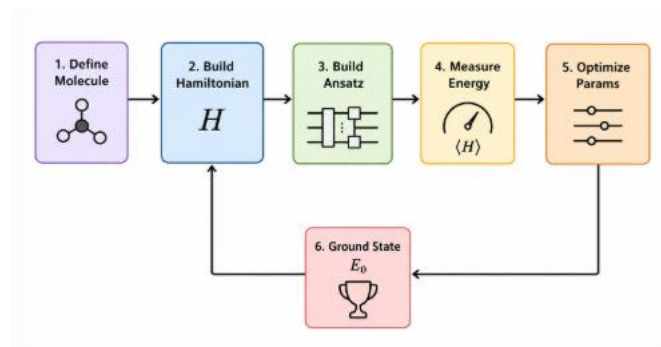
ground_energy = result.eigenvalue.real
optimal_params = result.optimal_parameters
error_mhartree = abs(ground_energy - EXACT_ENERGY) * 1000

print("\n" + "=" * 50)
print("VQE Results for H2")
print("=" * 50)
print(f"Ground state energy : {ground_energy:.6f} Hartree")
print(f"Exact (Full CI)      : {EXACT_ENERGY:.6f} Hartree")
print(f"Error                  : {error_mhartree:.3f} mHartree")
print(f"Optimal parameters    : {optimal_params}")
print("=" * 50)

```

vi) Complete Workflow Summary

Here is the complete picture of what we just did:



This is the elegance of VQE: the quantum computer does only what it is uniquely good at — evaluating quantum mechanical expectation values. All the problem setup, optimization, and result extraction happen on ordinary classical hardware.

7 Limitations of Current Quantum Hardware

VQE is often presented as the algorithm that brings quantum advantage to the near term. This is true in principle, but there are real challenges that limit what we can achieve today:

- **Noise and decoherence:** Real quantum hardware introduces errors in every gate. For VQE on a noisy device, these errors accumulate with circuit depth, degrading the energy estimate. Error mitigation techniques can help, but they do not eliminate the problem entirely.
- **Barren plateaus:** For deep ansatz circuits with many parameters, the energy landscape can become exponentially flat — gradients vanish and the optimizer makes no progress. This is called the barren plateau problem and is an active area of research.
- **Measurement overhead:** Estimating the energy requires measuring many Pauli terms, each requiring many circuit repetitions (shots). For large molecules, this measurement overhead can become prohibitive.
- **Ansatz expressibility:** If the true ground state lies outside the family of states that the ansatz can represent, VQE cannot find it no matter how well we optimize. Choosing the right ansatz for a given problem is still an open question.

Despite these challenges, VQE has been successfully demonstrated on real quantum hardware for small molecules. As quantum hardware improves — more qubits, lower error rates, better connectivity — the practical utility of VQE will grow significantly.

Philosophy of VQE

Rather than waiting for perfect quantum computers that do not yet exist, VQE embraces the limitations of today's hardware and uses classical computation to compensate. It is a practical, pragmatic, and deeply clever algorithm.

The key ideas to take away are:

- The variational principle guarantees that any energy estimate is an upper bound on the true ground state energy.
- The ansatz is a parameterized quantum circuit. Better ansätze lead to more accurate results.
- The classical optimizer drives the parameter updates. The quantum computer only evaluates energies.
- VQE is a hybrid algorithm. This hybrid nature is what makes it viable on today's noisy devices.

A Special Note to the Reader

This text was written to help you build an intuitive understanding of VQE, not to cover every mathematical detail.

It is a starting point for your exploration of quantum algorithms. As you learn more, try asking deeper questions:

- . Why does the algorithm work?
- . What are its limitations?
- . When does it outperform classical methods?
- . How is it implemented on real hardware?

The best way to learn quantum computing is to stay curious, experiment, and keep asking questions. Every algorithm you study will reveal a new perspective on how quantum systems can be used for computation.

REFERENCES

Original Paper

Peruzzo, A., McClean, J., Shadbolt, P., Yung, M. H., Zhou, X. Q., Love, P. J., Aspuru-Guzik, A., & O'Brien, J. L. (2014). A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 15(1), 4213.

Quantum Computing Foundations

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information* (10th anniversary ed.). Cambridge University Press.

Software & Documentation

Qiskit contributors. (2023). *Qiskit: An open-source framework for quantum computing*. Zenodo. <https://doi.org/10.5281/zenodo.2573505>

Qiskit Nature documentation. <https://qiskit-community.github.io/qiskit-nature/>

Qiskit Algorithms documentation. <https://qiskit-community.github.io/qiskit-algorithms/>